



UNIVERSIDAD DE LOS LAGOS

LABORATORIO 2

TALLER DE INGENIERÍA INFORMÁTICA

DEPARTAMENTO DE CIENCIAS DE LA INGENIERÍA
INGENIERÍA CIVIL EN INFORMÁTICA
OSORNO, CHILE

Matías Agüero

Alier Fres

Nicolás Guerrero

Francisco Núñez

4 de junio de 2026



5 años
UNIVERSIDAD ACREDITADA
Septiembre de 2021 - Septiembre de 2026
Gestión Institucional - Docencia de Pregrado
Investigación - Vinculación con el Medio
AVANZADA

www.ulagos.cl

Índice general

1. Introducción	3
2. Desarrollo	4
2.1. Construcción	4
2.2. Ejecución	5
2.3. Logs/Red	6
2.4. Prueba Funcional	6
3. Conclusión	9
4. Anexos	10
4.1. Enlaces del Proyecto	10
4.2. Configuración Docker	10
4.2.1. Archivo <code>docker-compose.yml</code>	10
4.2.2. Archivo <code>Dockerfile</code>	11

Índice de figuras

2.1. Proceso de construcción de la imagen app mediante Dockerfile.	5
2.2. Ejecución de contenedores y verificación de estado en la red.	5
2.3. Logs de inicio y configuración del contenedor MariaDB.	6
2.4. Interfaz principal del CRUD mostrando los registros actuales.	7
2.5. Formulario de actualización de datos de un registro existente.	7
2.6. Confirmación de eliminación de un registro en la base de datos.	8
2.7. Visualización de la tabla sin registros demostrando la limpieza total.	8

Capítulo 1

Introducción

En el escenario del desarrollo de software e infraestructura moderna, el uso de contenedores se ha consolidado como una estrategia clave para resolver los problemas clásicos de inconsistencia de entornos y facilitar el despliegue de aplicaciones. El presente informe técnico detalla el proceso de diseño e implementación de un stack web dinámico utilizando Docker, desarrollado en el marco del Laboratorio Práctico N°2. El propósito central de esta experiencia práctica consiste en desplegar una aplicación web funcional capaz de realizar las cuatro operaciones esenciales de gestión de datos (CRUD: Crear, Leer, Actualizar y Borrar) dentro de un ecosistema controlado y seguro.

Para dar cumplimiento a los requisitos técnicos establecidos, la arquitectura se estructuró mediante la herramienta `docker-compose`, dividiendo el sistema en dos microservicios principales: un contenedor para la aplicación web configurado mediante un archivo `Dockerfile` personalizado (PHP) y un contenedor para el motor de base de datos relacional utilizando una imagen oficial (MariaDB). A lo largo de este documento, se expondrá la evidencia técnica del despliegue, analizando aspectos críticos de la infraestructura como el aislamiento de los servicios dentro de una red virtual privada de Docker y la configuración de volúmenes para garantizar la persistencia de los datos de forma independiente al ciclo de vida de los contenedores.

Capítulo 2

Desarrollo

A continuación, se presenta la evidencia técnica del levantamiento de la infraestructura, siguiendo el orden lógico del despliegue en la consola del entorno Debian.

2.1. Construcción

El ciclo de vida del despliegue comienza con la fase de construcción de las imágenes personalizadas. Para ello, se ejecutó el comando `docker compose build` en la consola de Linux. Este comando se encarga de interpretar las instrucciones declaradas en los archivos `Dockerfile` de cada servicio.

Durante este proceso, el Docker Daemon descarga las imágenes base desde el registro oficial, ejecuta de forma secuencial cada capa de configuración (instalación de dependencias, copia de código fuente y exposición de puertos) y genera los artefactos locales listos para su ejecución. En la Figura 2.1 se observa el output de la consola reflejando la correcta finalización del proceso sin errores de compilación.

```

[+] Building 54.4s (8/8) FINISHED
=> [internal] load local bake definitions 0.0s
=> reading from stdin 0.0s
=> [internal] load build definition from Dockerfile 0.1s
=> transferring dockerfile: 110B 0.0s
=> [internal] load metadata for docker.io/library/php:8.2-apache 2.0s
=> [internal] load metadata for docker.io/library/php:8.2-apache 0.1s
=> [internal] load metadata for docker.io/library/php:8.2-apache 0.0s
=> [1/2] FROM docker.io/library/php:8.2-apache:sha256:affc043fbd9aca9a6394a71d162726fcd9e64bea840a3b04f925b6130858dd 21.7s
=> resolve docker.io/library/php:8.2-apache:sha256:affc043fbd9aca9a6394a71d162726fcd9e64bea840a3b04f925b6130858dd 0.0s
=> sha256:44f470ee5f540c1e0371a1e0b0a0d1e0c0557f48a0d7f9e0a63b0dca1308 / 220 0.3s
=> sha256:759ec9d2949ea70ba880f8fe74c0b0e95caae1f670f2b0e6a968923c11ca 892B / 892B 0.3s
=> sha256:ffe0fb363206203fc595e8a0e27574f85e091fad8d4099427e088d029e85f 249B / 249B 0.4s
=> sha256:9a2c3b28bcf76a00ef150b921dec85804077b07e7d5636d765286d0a8080e 255B / 255B 0.0s
=> sha256:1b4118522fd02b7c95bae8f530b1a1b08113192ebc1ee973cd05b061e1ff 2.4kB / 2.4kB 0.3s
=> sha256:e094c55ba5ab821149fcb0c67d1f1ae38892d150ba0980a069ceda8b20188 11.4kB / 11.4kB 2.1s
=> sha256:e4885f58575c91f8ad8120f0557959fb195a5cf63174e2430877af9840c5211 488B / 488B 0.3s
=> sha256:72950e9a0a870c762c080399a3c1eeef79330e2780eb43a5a1632baef8 12.3kB / 12.3kB 2.1s
=> sha256:bc3476dda7999ef33bfb0e9f621ed071892e06223ef0a8b4649f54f0ab69e 488B / 488B 0.3s
=> sha256:d067a157ac10dbce34216e5c0a2b275ede97675081157c6e42b8d0c87805e5 435B / 435B 0.3s
=> sha256:f820f7789dfe0d1ec9e5d86920870b0f8af7eabbc524375c958ca66132a46 4.2kB / 4.2kB 1.1s
=> sha256:5c08f9590e7701345e07c0d0c5e1da24ac7847b2c4f62356638e849f25b 226B / 226B 0.3s
=> sha256:487e97156f0056dc9518746757ab1114f0ea303f54e70f2a5bca1a6e53fc39 117.8kB / 117.8kB 6.7s
=> sha256:54eb7071422ecd73d7d21a7467f66fc93bb19f4dc9328ea86026a7e781f57 225B / 225B 0.3s
=> extracting sha256:54eb7071422ecd73d7d21a7467f66fc93bb19f4dc9328ea86026a7e781f57 0.2s
=> extracting sha256:487e97156f0056dc9518746757ab1114f0ea303f54e70f2a5bca1a6e53fc39 9.9s
=> extracting sha256:5c08f9590e7701345e07c0d0c5e1da24ac7847b2c4f62356638e849f25b 0.0s
=> extracting sha256:f820f7789dfe0d1ec9e5d86920870b0f8af7eabbc524375c958ca66132a46 0.5s
=> extracting sha256:d067a157ac10dbce34216e5c0a2b275ede97675081157c6e42b8d0c87805e5 0.1s
=> extracting sha256:bc3476dda7999ef33bfb0e9f621ed071892e06223ef0a8b4649f54f0ab69e 0.0s
=> extracting sha256:72950e9a0a870c762c080399a3c1eeef79330e2780eb43a5a1632baef8 0.4s
=> extracting sha256:54eb7071422ecd73d7d21a7467f66fc93bb19f4dc9328ea86026a7e781f57 0.9s
=> extracting sha256:e094c55ba5ab821149fcb0c67d1f1ae38892d150ba0980a069ceda8b20188 1.3s
=> extracting sha256:1b4118522fd02b7c95bae8f530b1a1b08113192ebc1ee973cd05b061e1ff 0.0s
=> extracting sha256:9a2c3b28bcf76a00ef150b921dec85804077b07e7d5636d765286d0a8080e 0.0s
=> extracting sha256:ffe0fb363206203fc595e8a0e27574f85e091fad8d4099427e088d029e85f 0.9s
=> extracting sha256:759ec9d2949ea70ba880f8fe74c0b0e95caae1f670f2b0e6a968923c11ca 0.0s
=> extracting sha256:4f4fb700a5f46d1cf02571aeb0b9a0c1e0cd5577484ad75e58dc38e8ac1 0.0s
[2/2] RUN docker-php-ext-install mysqli pdo pdo_mysql 29.2s
=> exporting to image 0.7s
=> exporting layers 0.3s
=> exporting manifest sha256:ccc90c4317659fd6b7ad5be876da883e79e870e9023daf7143acalcfbef8 0.0s
=> exporting config sha256:98792d29492c3a8ba7ee116fa7c1194f216d083c394c72e1cf61413eb 0.0s
=> exporting attestation manifest sha256:79742b948976ea380ba8a5e0a3db428a6e7e5a25c4741419fb7024bf47c6c4 0.1s
=> exporting manifest list sha256:5527cfa1ad663270d4f4b87a04bd3a7619c12b0da573c3db93622996d6b2c01 0.0s
=> naming to docker.io/library/taller-docker-app:latest 0.0s
=> unpacking to docker.io/library/taller-docker-app:latest 0.1s
=> resolving provenance for metadata file 0.0s
[+] build 1/1
Image taller-docker-app built 54.5s
root@debian:/taller-docker#

```

Figura 2.1: Proceso de construcción de la imagen app mediante Dockerfile.

2.2. Ejecución

Se utilizó el comando `docker compose up -d` para levantar los servicios en segundo plano, seguido del comando `docker ps`. La captura demuestra la correcta orquestación de la infraestructura, confirmando que tanto el contenedor de la aplicación web (mapeado al puerto local 8080) como el de la base de datos se encuentran en estado activo (Up) e interactuando en la red virtual `taller-red`.

```

root@debian:/taller-docker# docker compose up -d
[+] up 15/15
Network taller-docker-taller-red Created
Volume taller-docker_db_data Created
Container taller-docker-db-1 Started
Container taller-docker-app-1 Started
root@debian:/taller-docker# docker ps
CONTAINER ID   IMAGE               COMMAND                  STATUS      PORTS                               NAMES
a02facaa594    taller-docker-app  "docker-php-entrypoint..." 4 seconds ago Up 3 seconds 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp taller-docker-app-1
f78925a5971    mariadb:10.5      "docker-entrypoint.sh..." 5 seconds ago Up 3 seconds 3306/tcp                  taller-docker-db-1
87f1ab9f5dc    odoo:18.0         "entrypoint.sh odoo..." 22 hours ago Up 22 minutes 0.0.0.0:8069->8069/tcp, [::]:8069->8069/tcp, 8071-8072/tcp odoo-taller-web-1
a0bba169f45    ps:alpine          "docker-entrypoint.sh..." 22 hours ago Up 22 minutes 5432/tcp                  odoo-taller-db-1
root@debian:/taller-docker#

```

Figura 2.2: Ejecución de contenedores y verificación de estado en la red.

2.3. Logs/Red

Para comprobar el correcto inicio del servicio MariaDB y la lectura del script de inicialización (`init.sql`), se inspeccionaron los registros mediante el comando `docker compose logs db`. La imagen verifica que el motor de base de datos se inicializó exitosamente, asignó los permisos correspondientes al usuario y se encuentra a la escucha de conexiones a través del puerto 3306 interno.

```

root@192.168.1.94:22 - Bitvise xterm
db-1 2026-06-04 0:52:22 0 [Note] InnoDB: log sequence number 45568; transaction id 14
db-1 2026-06-04 0:52:22 0 [Note] Plugin 'FEEDBACK' is disabled.
db-1 2026-06-04 0:52:22 0 [Note] Plugin 'wsrep-provider' is disabled.
db-1 2026-06-04 0:52:24 0 [Note] mariadbd: Event Scheduler: Loaded 0 events
db-1 2026-06-04 0:52:24 0 [Note] mariadbd: ready for connections.
db-1 2026-06-04 0:52:24 0 [Note] mariadbd: Version: '12.3.2-MariaDB-ubu2404' socket: '/run/mysqld/mysqld.sock' port: 0 mariadb.org binary distribution
db-1 2026-06-04 00:52:25+00:00 [Note] [Entrypoint]: Temporary server started.
db-1 2026-06-04 00:52:25+00:00 [Note] [Entrypoint]: Creating database crud_db
db-1 2026-06-04 00:52:25+00:00 [Note] [Entrypoint]: Creating user usuario_db
db-1 2026-06-04 00:52:25+00:00 [Note] [Entrypoint]: Giving user usuario_db access to schema crud_db
db-1 2026-06-04 00:52:25+00:00 [Note] [Entrypoint]: Securing system users (equivalent to running mysql_secure_installation)
db-1 2026-06-04 00:52:26+00:00 [Note] [Entrypoint]: Stopping temporary server
db-1 2026-06-04 0:52:26 0 [Note] mariadbd (initiated by: unknown): Normal shutdown
db-1 2026-06-04 0:52:26 0 [Note] InnoDB: FTS optimize thread exiting.
db-1 2026-06-04 0:52:26 0 [Note] InnoDB: Starting shutdown...
db-1 2026-06-04 0:52:26 0 [Note] InnoDB: Dumping buffer pool(s) to /var/lib/mysql/ib_buffer_pool
db-1 2026-06-04 0:52:26 0 [Note] InnoDB: Buffer pool(s) dump completed at 260604 0:52:26
db-1 2026-06-04 0:52:26 0 [Note] InnoDB: Removed temporary tablespace data file: "/ibtmp1"
db-1 2026-06-04 0:52:26 0 [Note] Shutdown completed; log sequence number 45568; transaction id 15
db-1 2026-06-04 0:52:26 0 [Note] mariadbd: Shutdown complete
db-1 2026-06-04 00:52:26+00:00 [Note] [Entrypoint]: Temporary server stopped
db-1 2026-06-04 00:52:26+00:00 [Note] [Entrypoint]: MariaDB init process done. Ready for start up.
db-1 2026-06-04 0:52:26 0 [Note] Starting MariaDB 12.3.2-MariaDB-ubu2404 source revision 9f98f82b14a9b939834281672b6d0cf965db69a3 server_uid Pbz2QwuOuCuTsrXAerJQ4+B8Csw= as process 1
db-1 2026-06-04 0:52:26 0 [Note] InnoDB: Compressed tables use zlib 1.3
db-1 2026-06-04 0:52:26 0 [Note] InnoDB: Number of transaction pools: 1
db-1 2026-06-04 0:52:26 0 [Note] InnoDB: Using crc32 + pclmulqdq instructions
db-1 2026-06-04 0:52:26 0 [Warning] mariadbd: io_uring_queue_init() failed with EPERM: sysctl kernel.io_uring_disabled has the value 2, or 1 and the user of the process is not a member of sysctl kernel.io_uring_group. (see man 2 io_uring_setup).
db-1 2026-06-04 0:52:26 0 [Note] InnoDB: Using Linux native AIO
db-1 2026-06-04 0:52:26 0 [Note] InnoDB: innodb_buffer_pool_size_max=8388608m, innodb_buffer_pool_size=128m
db-1 2026-06-04 0:52:26 0 [Note] InnoDB: Completed initialization of buffer pool
db-1 2026-06-04 0:52:26 0 [Note] InnoDB: File system buffers for log disabled (block size=512 bytes)
db-1 2026-06-04 0:52:26 0 [Note] InnoDB: End of log at LSN=45568
db-1 2026-06-04 0:52:26 0 [Note] InnoDB: Opened 3 undo tablespaces
db-1 2026-06-04 0:52:26 0 [Note] InnoDB: 128 rollback segments in 3 undo tablespaces are active.
db-1 2026-06-04 0:52:26 0 [Note] InnoDB: Setting file "/ibtmp1" size to 12.000MiB. Physically writing the file full; Please wait ...
db-1 2026-06-04 0:52:26 0 [Note] InnoDB: File "/ibtmp1" size is now 12.000MiB.
db-1 2026-06-04 0:52:26 0 [Note] InnoDB: log sequence number 45568; transaction id 14
db-1 2026-06-04 0:52:26 0 [Note] Plugin 'FEEDBACK' is disabled.
db-1 2026-06-04 0:52:26 0 [Note] Plugin 'wsrep-provider' is disabled.
db-1 2026-06-04 0:52:26 0 [Note] InnoDB: Loading buffer pool(s) from /var/lib/mysql/ib_buffer_pool
db-1 2026-06-04 0:52:26 0 [Note] InnoDB: Buffer pool(s) load completed at 260604 0:52:26
db-1 2026-06-04 0:52:29 0 [Note] Server socket created on IP: '0.0.0.0', port: '3306'.
db-1 2026-06-04 0:52:29 0 [Note] Server socket created on IP: '::', port: '3306'.
db-1 2026-06-04 0:52:29 0 [Note] mariadbd: Event Scheduler: Loaded 0 events
db-1 2026-06-04 0:52:29 0 [Note] mariadbd: ready for connections.
db-1 2026-06-04 0:52:29 0 [Note] mariadbd: Version: '12.3.2-MariaDB-ubu2404' socket: '/run/mysqld/mysqld.sock' port: 3306 mariadb.org binary distribution
root@debian:~/taller-docker#

```

Figura 2.3: Logs de inicio y configuración del contenedor MariaDB.

2.4. Prueba Funcional

Finalmente, se accede a la aplicación web a través del navegador apuntando al puerto mapeado (8080). La captura demuestra la interfaz del menú principal cargando de manera óptima, conectada a la base de datos y lista para ejecutar la gestión de registros (CRUD) solicitada en los requisitos técnicos del laboratorio.

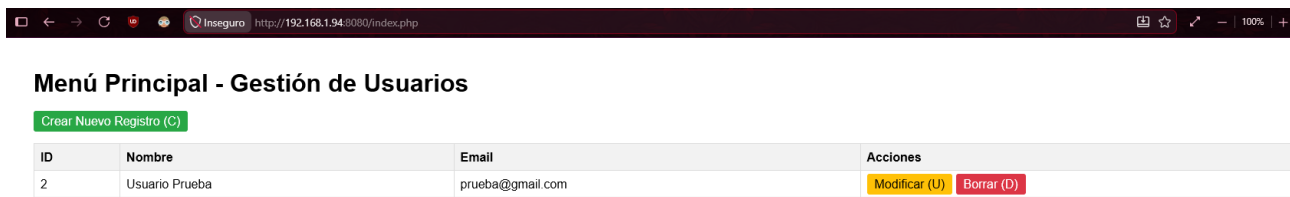


Figura 2.4: Interfaz principal del CRUD mostrando los registros actuales.



Figura 2.5: Formulario de actualización de datos de un registro existente.

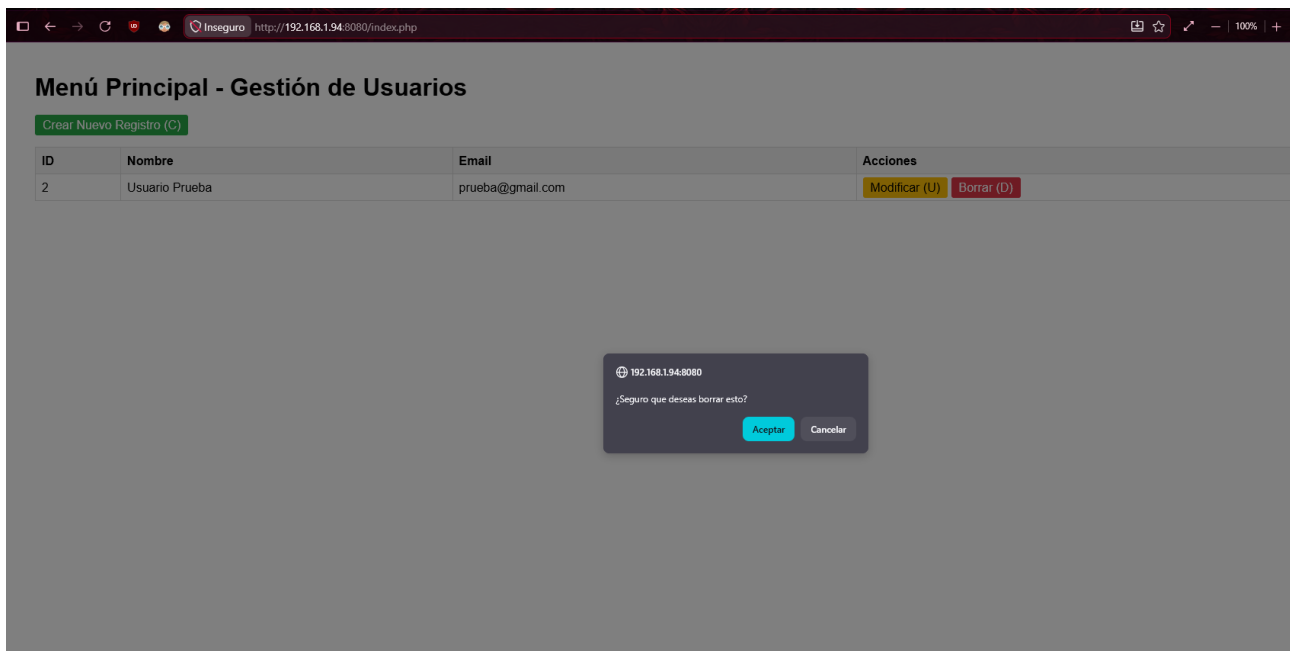


Figura 2.6: Confirmación de eliminación de un registro en la base de datos.

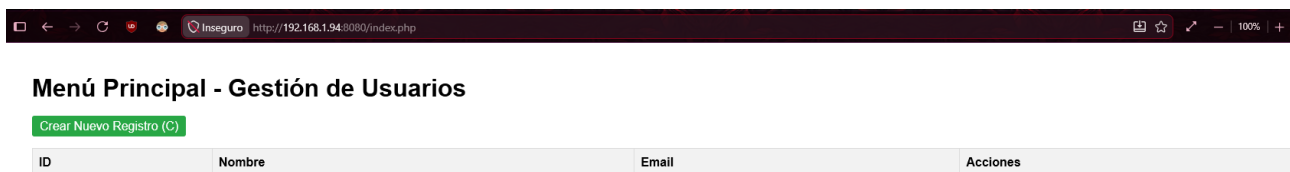


Figura 2.7: Visualización de la tabla sin registros demostrando la limpieza total.

Capítulo 3

Conclusión

Un aspecto crítico evaluado fue la persistencia de datos. Mediante las asignaciones y configuraciones de volúmenes en Docker, se demostró directamente que la capa de datos mantiene su integridad de forma totalmente independiente a la volatilidad, detención o destrucción eventual de los contenedores. Esto demuestra el valor de abstraer el almacenamiento lógico del ciclo de vida del servicio, un estándar indispensable en entornos de producción reales. De igual forma, el aislamiento de los contenedores dentro de una red virtual privada (`taller-red`) permitió comprender cómo mitigar vectores de riesgo, limitando la exposición del motor de base de datos y forzando a que toda interacción ocurra de manera segura y exclusiva a través del servidor web

Finalmente, el correcto funcionamiento de las cuatro operaciones fundamentales del sistema CRUD, evidenciado en la etapa de pruebas en el navegador, validó no solo la conectividad y resolución de nombres interna del stack, sino también la viabilidad de Docker como un ecosistema eficiente para el despliegue de aplicaciones. Los objetivos del laboratorio fueron alcanzados, dándole al equipo experiencia técnica esencial en infraestructura de programa, automatización y buenas prácticas de desarrollo modular.

Capítulo 4

Anexos

4.1. Enlaces del Proyecto

Todo el código fuente, la configuración de contenedores y los scripts de base de datos se encuentran respaldados con control de versiones. A continuación, se detallan los enlaces correspondientes:

- **Repositorio en GitHub:** <https://github.com/KriegerN/taller-lab-2>
- **Link del documento en Overleaf:** <https://es.overleaf.com/read/jnknbpbgxxdb#5aeb9d>

4.2. Configuración Docker

A continuación se detallan los archivos de configuración utilizados para la orquestación y construcción de los contenedores de este laboratorio.

4.2.1. Archivo `docker-compose.yml`

```
1 services:
2   app:
3     build: ./app
4     ports:
5       - "8080:80"
6     volumes:
7       - ./app:/var/www/html
8     networks:
9       - taller-red
10    depends_on:
11      - db
12
13  db:
14    image: mariadb:10.5
15    restart: always
16    environment:
```

```
17     MYSQL_ROOT_PASSWORD: rootpassword
18     MYSQL_DATABASE: crud_db
19     MYSQL_USER: usuario_db
20     MYSQL_PASSWORD: password_db
21     volumes:
22     - db_data:/var/lib/mysql
23     - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql
24     networks:
25     - taller-red
26
27 networks:
28     taller-red:
29         driver: bridge
30
31 volumes:
32     db_data:
```

4.2.2. Archivo Dockerfile

```
1 FROM php:8.2-apache
2 RUN docker-php-ext-install mysqli pdo pdo_mysql
3 EXPOSE 80
```